Software Testing and Reliability

Software Testing → A process of **evaluating a software system** to check whether it meets requirements and works correctly under different conditions.

Software Reliability → The probability that software will operate **without failure** for a specified time under given conditions.

Software Testing

Software Testing is a systematic process of **executing a program with the intent to find errors** and ensure the software meets **functional and non-functional requirements**.

Objectives of Testing

- To detect defects early.
- To ensure software meets user requirements.
- To improve quality, performance, security.
- To provide confidence before release.

Levels of Testing

Testing is not done in a single step. It is performed at different **levels**, starting from individual components up to the whole system. Each level has its **objectives, scope, and methods**.

The four main levels are:

1. **Unit Testing**
2. **Integration Testing**
3. **System Testing**
4. **Acceptance Testing**

1. Unit Testing

Unit Testing checks the **smallest testable parts** of software, usually functions, classes, or modules, to verify they work as expected.

Purpose

- Validate correctness of **individual units**.
- Detect errors early in development.
- Ensure code behaves as per design.

Who Performs?

- Developers (often with automated tools).

Techniques

- **White-box testing** → path coverage, condition coverage.
- Use of **stubs** and **drivers** when modules depend on others.

Tools

- JUnit (Java), NUnit (.NET), pytest (Python).

Example: Testing a function that calculates the interest in a banking app.

2. Integration Testing

Integration Testing ensures that **modules or units work together** when combined.

Purpose

- Detect interface defects (mismatched data types, incorrect API calls).
- Ensure communication between modules is correct.

Who Performs?

- Developers or testers.

Approaches

1. **Top-Down Integration** → Start with main control module, integrate downwards using *stubs*.
2. **Bottom-Up Integration** → Start with lower modules, combine upwards using *drivers*.
3. **Big-Bang Integration** → All modules integrated at once (risk of chaos).
4. **Sandwich/Hybrid Integration** → Combination of top-down and bottom-up.

Example: Checking whether the login module passes correct user data to the dashboard module.

3. System Testing

System Testing tests the **entire application** as a whole, validating both functional and non-functional requirements.

Purpose

- Verify that the system meets its specifications.
- Test in an environment similar to production.

Who Performs?

- Independent testing team (QA).

Types

- **Functional Testing** → Requirements validation.
- **Performance Testing** → Load, stress, scalability.
- **Security Testing** → Vulnerability checks, penetration tests.
- **Usability Testing** → User-friendliness.
- **Compatibility Testing** → Different browsers, devices, OS.

Example: Testing an e-commerce website with complete order processing (search, cart, payment, confirmation).

4. Acceptance Testing

Definition

Acceptance Testing validates the software **against user/business requirements** and determines if it is ready for release.

Purpose

- Ensure the software solves the customer's problem.
- Provide confidence before deployment.

Who Performs?

- End-users, clients, or QA team on behalf of the client.

Types

1. **Alpha Testing** → Done at developer's site, with limited user involvement.
2. **Beta Testing** → Done at customer's site, with real-world users.
3. **User Acceptance Testing (UAT)** → Performed by end-users.

Example: A bank verifies its new online transaction system before releasing it to customers.

Level	Scope	Focus	Performed By
Unit Testing	Single module	Correctness of code	Developers
Integration Test	Interfaces	Module interaction	Dev/QA
System Testing	Whole system	Requirements	QA Team
Acceptance Test	Real use case	Business needs	Users/Clients

Types of Software Testing

Software Testing types are generally divided into **Functional Testing** and **Non-Functional Testing**, but there are also **Maintenance Testing** and **Specialized Testing** categories.

1. Functional Testing

Functional testing checks **what the system does** — i.e., does the software behave as per requirements?

Main Types

a. Unit Testing

- Smallest testable part (functions, methods).
- Ensures correctness of individual components.

b. Integration Testing

- Verifies interaction between modules.
- Example: Login module passing data to dashboard module.

c. System Testing

- Validates the complete system as per requirements.
- Example: An e-commerce site from product search to payment.

d. Regression Testing

- Ensures new code does not break old functionality.
- Done after bug fixes or enhancements.

e. Sanity Testing

- Quick test of specific functionality after small changes.

f. Smoke Testing

- Initial "build verification testing".
- Checks if the basic system runs without crashing.

g. User Acceptance Testing (UAT)

- Performed by end-users.
- Confirms system meets **business needs**.

2. Non-Functional Testing

Non-functional testing checks **how well the system works** — i.e., performance, usability, reliability, etc.

Main Types

a. Performance Testing

- Evaluates speed, response time, and resource usage.
- **Load Testing** → System under expected load.
- **Stress Testing** → Extreme load beyond limits.
- **Soak (Endurance) Testing** → Long-duration usage.

b. Security Testing

- Identifies system vulnerabilities.
- Includes penetration testing, ethical hacking.
- Example: Checking if password data is encrypted.

c. Usability Testing

- Ensures software is **user-friendly**.
- Example: Simple navigation, readable fonts.

d. Compatibility Testing

- Runs on multiple browsers, devices, OS.
- Example: Website works on Chrome, Firefox, Safari.

e. Scalability Testing

- Measures ability to handle growing workload.
- Example: E-commerce site handling Black Friday traffic.

f. Reliability Testing

- Ensures software works without failure for a period.

g. Portability Testing

- Checks if software runs on different environments.
- Example: Mobile app running on Android & iOS.

3. Maintenance Testing

Done during **software maintenance** (after release).

- **Regression Testing** → Ensures updates don't break old features.
- **Re-testing (Confirmation Testing)** → Ensures fixed bug really works.

Specialized Testing

Advanced/specific testing types:

1. Alpha Testing

- At developer's site, internal testing with limited users.

2. Beta Testing

- Real-world testing by actual customers before official release.

3. Ad-hoc Testing

- Random, unstructured testing without documentation.

4. Exploratory Testing

- Tester explores software while learning it.

5. Monkey Testing

- Random inputs, stress testing software unexpectedly.

6. **Mutation Testing**

- Injecting small changes (mutants) in code to check if test cases detect them.

7. **A/B Testing**

- Comparing two versions (common in web apps, marketing).

Software Testing Techniques

Testing techniques are structured **methods used to design test cases** and ensure that the system is tested effectively.

They are broadly divided into:

1. **Black-Box Testing Techniques**
2. **White-Box Testing Techniques**
3. **Gray-Box Testing Techniques**

1. Black-Box Testing Techniques

Black-box testing focuses on the **functionality** of the system without looking at internal code.

The tester only cares about **inputs and expected outputs**.

1.1 Equivalence Partitioning (EP)

- Divide input data into **equivalence classes** where all values should behave similarly.
- Choose one representative from each class.

Example:
If valid age is 18–60 →

- Class 1: below 18 (invalid).
- Class 2: 18–60 (valid).
- Class 3: above 60 (invalid).

1.2 Boundary Value Analysis (BVA)

- Errors often occur at boundaries.
- Test values just **below, at, and above boundaries**.

Example:

Age input 18–60 → test **17, 18, 60, 61**.

1.3 Decision Table Testing

- Used when system behavior depends on **combinations of conditions**.
- Represents inputs/conditions and outputs/actions in a **table**.

Example:
Loan approval depends on **salary & credit score**.

Salary > 50k	**Credit Good**	**Action**
Yes	Yes	Approve
Yes	No	Review
No	Yes	Review
No	No	Reject

1.4 State Transition Testing

- Used when system behavior changes with **states and events**.
- Test all possible **state changes**.

Example:
ATM → States: Idle → PIN Entered → Transaction → Idle.

1.5 Use Case Testing

- Based on real-world **user scenarios**.
- Each **use case = test case**.

Example: Shopping cart → Add product → Checkout → Payment → Confirmation.

2. White-Box Testing Techniques

White-box testing requires **knowledge of internal code logic**. It ensures that **all paths, conditions, and loops** are tested.

2.1 Statement Coverage

- Every **line of code** must be executed at least once.

2.2 Branch Coverage (Decision Coverage)

- Every **true/false branch** of decision points must be tested.

2.3 Condition Coverage

- Each **boolean condition** in a decision must be tested for both true/false.

2.4 Path Coverage

- Ensures **all possible execution paths** are tested.
- Very thorough but expensive.

2.5 Loop Testing

- Special focus on loops → test **0, 1, many iterations**.

3. Gray-Box Testing Techniques

Combination of **black-box and white-box**. Tester has **partial knowledge** of the system internals.

Examples

- Testing a web app with knowledge of DB schema.
- API testing with knowledge of request/response formats.
- Security testing with partial knowledge of source code.

Software Reliability

Software Reliability is the probability that software will function correctly without failure under specified conditions for a given time period.

Reliability = Correctness + Robustness + Availability.

Factors Affecting Reliability

- Complexity of software (more LOC → more bugs).
- Development process quality.
- Testing effectiveness.
- Operating environment (hardware, users, network).
- Fault tolerance mechanisms.

Reliability Metrics

Reliability is one of the most important non-functional requirements of software. Since software never "wears out" like hardware, we need mathematical and statistical measures to quantify how reliable a system is. These measures are called Reliability Metrics.

They help in:

- Predicting software performance.
- Estimating downtime.
- Comparing different systems.
- Making design and maintenance decisions.

1. MTTF (Mean Time to Failure)

- Definition: The average time a system runs before it fails.
- Formula:

$$MTTF = \text{Total operational time} / \text{Number of failures}$$

- Example:
  If software runs for 1,000 hours and fails 5 times:

  $$MTTF=1000/5=200$$

- Use: For non-repairable systems (embedded software, appliances).

2. MTTR (Mean Time to Repair)

- The average time taken to fix a failure and restore the system.
- Formula:

  $$MTTR=\text{Total repair time}/\text{Number of repairs}$$

- Example:
  If 5 failures took 10, 8, 12, 6, and 14 hours to fix:

  $$MTTR=(10+8+12+6+14)/5=10 \text{ hours}$$

- Use: Helps estimate system downtime.

3. MTBF (Mean Time Between Failures)

- The expected time between two successive failures.
- Formula:

  $$MTBF=MTTF+MTTR$$

- Example:
  If MTTF = 200 hours and MTTR = 10 hours:

  $$MTBF=200+10=210 \text{ hours}$$

- Use: Commonly used in reliable systems that can be repaired.

4. Failure Rate (λ)

- Definition: The frequency of system failures per unit time.
- Formula:

  $$\lambda = 1/MTTF$$

- Example:
  If MTTF = 200 hours:

  $$\lambda = 1/200 = 0.005 \text{ failures/hour}$$

- Use: Indicates how often a system fails.

5. Reliability Function (R(t))

- Probability that the system will operate without failure over time t.
- Formula:

  $$R(t)=e^{-\lambda t}$$

- Example:
  If $\lambda = 0.005$ failures/hour, probability of running for 100 hours:

  $$R(100) = e^{-0.005\times100} = e^{-0.5} \approx 0.6065$$

→ The system has 60.65% chance of surviving 100 hours without failure.

6. Availability (A)

- Probability that a system is operational at any given time.
- Formula:

  $$A = MTTF/(MTTF+MTTR)$$

- Example:
  If MTTF = 200 hours and MTTR = 10 hours:

  $$A = 200/(200+10) = 0.952 \text{ or } 95.2\%$$

- Use: Critical for 24/7 services (banking, cloud, telecom).

7. Defect Density

- Number of defects per unit size of software (e.g., per KLOC).
- Formula:

  Defect Density=Total defects/KLOC (thousand lines of code)

- Example:
  If 40 defects are found in 20 KLOC:

  $$\text{Defect Density} = 40/20 = 2 \text{ defects/KLOC}$$

- Use: Compares code quality between modules/projects.

8. Fault Density

- Definition: Number of faults per unit of test cases executed.
- Formula:

  $$\text{Fault Density} = \text{Faults detected}/\text{Test cases executed}$$

9. Service Reliability Metrics

- Downtime per year = Hours system is unavailable.
- Uptime percentage (commonly used in SLAs):

  $$\text{Uptime} = [(\text{Total time} - \text{Downtime})/\text{Total time}] \times 100$$

  - 99% uptime → 3.65 days downtime per year.
  - 99.9% uptime → 8.76 hours downtime per year.
  - 99.99% uptime → 52.6 minutes downtime per year.

Classification of Reliability Metrics

1. Time-based metrics → MTTF, MTTR, MTBF, λ, R(t).
2. Defect-based metrics → Defect density, Fault density.
3. Availability-based metrics → Availability (A), Uptime percentage.
4. Customer-based metrics → SLA compliance, Mean User Downtime.

Applications of Reliability Metrics

- Designing reliable systems (aviation, healthcare, finance).
- Comparing vendors (based on SLA uptime).
- Evaluating testing effectiveness.
- Predicting failures in maintenance planning.
- Supporting reliability growth models (Musa, Goel-Okumoto).